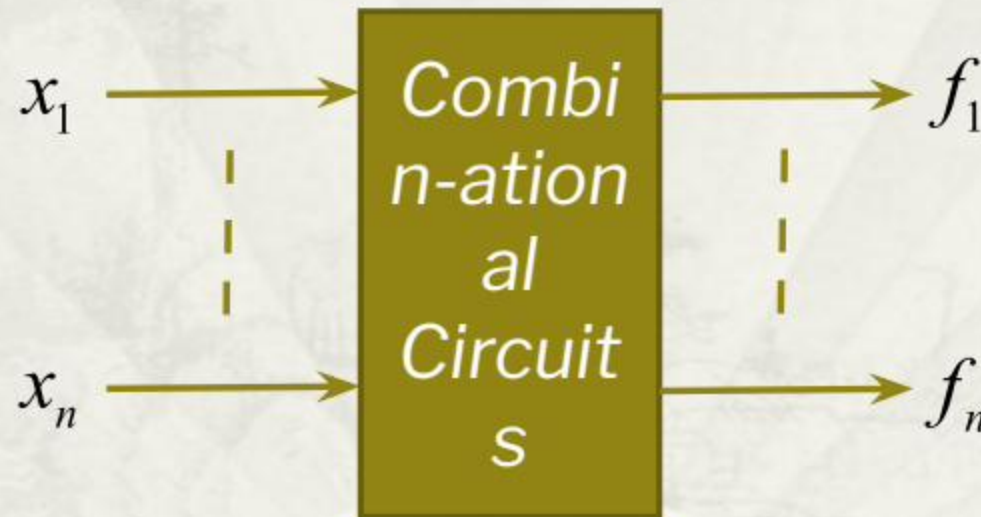


CH3 Combinational Logic Circuits

3.1 Features of Combinational Circuits

- ▣ Instantaneous response
 - ▣ Non-memorial circuits
 - ▣ Outputs determined only by current inputs
 - ▣ Consisting of gates or building blocks



$$f_i = f(x_1, x_2, \dots, x_n) \quad i = 1, 2, \dots, n$$

3.2 Issues on Combinational Circuits

- **Analysis** given circuit diagrams
- **Design** and implementation according to requirements
- Widely used **building blocks**: modular design

3.3 **Analysis** of basic Combinational Circuits

3.3.1. Approach

- Step 1: find logical expressions for outputs
- Step 2: minimizations
- Step 3: build the truth table(or K-map)
- Step 4: describe functionalities verbally.

3.3.2. Examples

- 1. Find F level-by-level

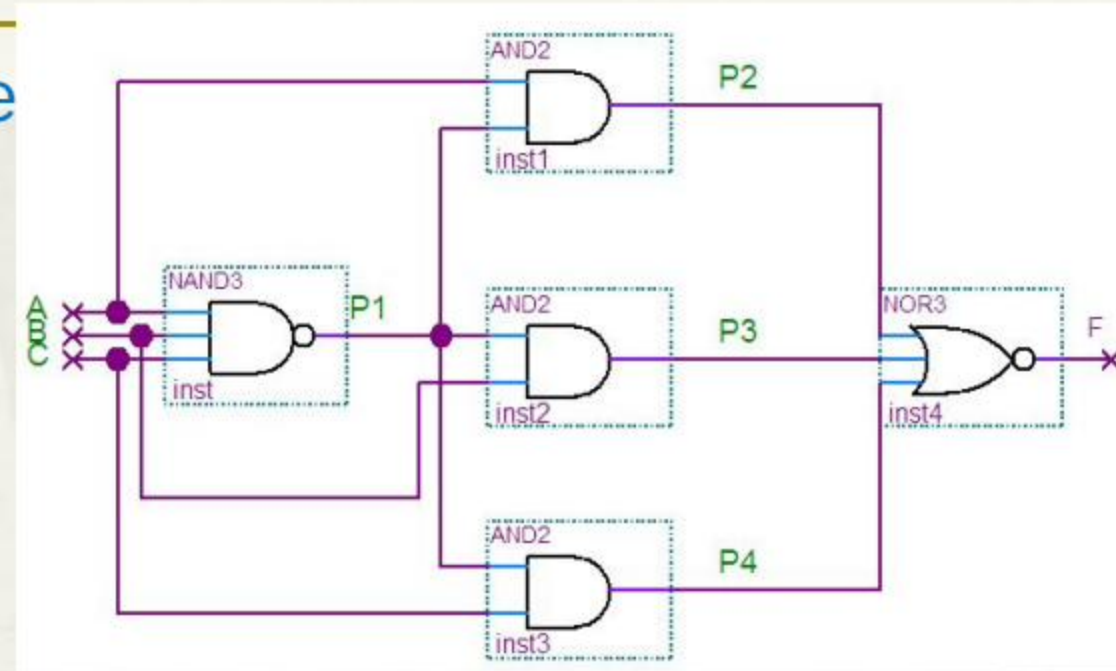
$$P_1 = \overline{ABC}$$

$$P_2 = A \cdot P_1 = A \cdot \overline{ABC}$$

$$P_3 = B \cdot P_1 = B \cdot \overline{ABC}$$

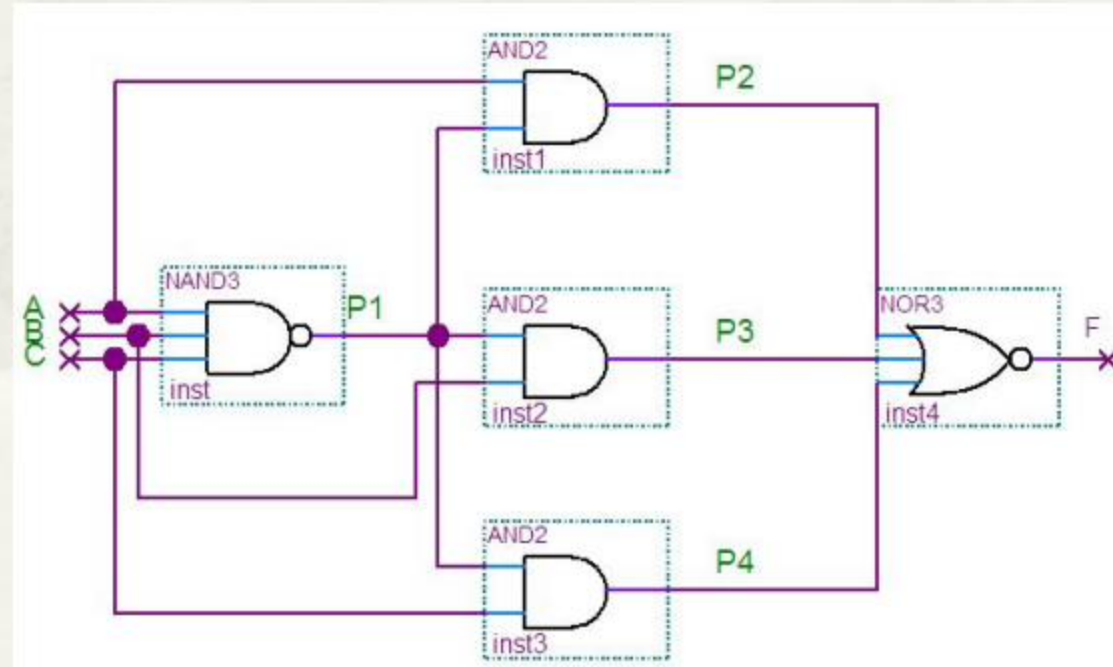
$$P_4 = C \cdot P_1 = C \cdot \overline{ABC}$$

$$F = \overline{P_2 + P_3 + P_4} = \overline{A \cdot \overline{ABC} + B \cdot \overline{ABC} + C \cdot \overline{ABC}}$$



□ 2. Simplify F

$$\begin{aligned} F &= \overline{P_2 + P_3 + P_4} = \overline{A \cdot \overline{ABC} + B \cdot \overline{ABC} + C \cdot \overline{ABC}} \\ &= \overline{(A + B + C) \overline{ABC}} \\ &= \overline{A + B + C} + \overline{\overline{ABC}} \\ &= \overline{\overline{ABC}} + ABC \end{aligned}$$



- 3. Construct the truth table

A	B	C	F
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

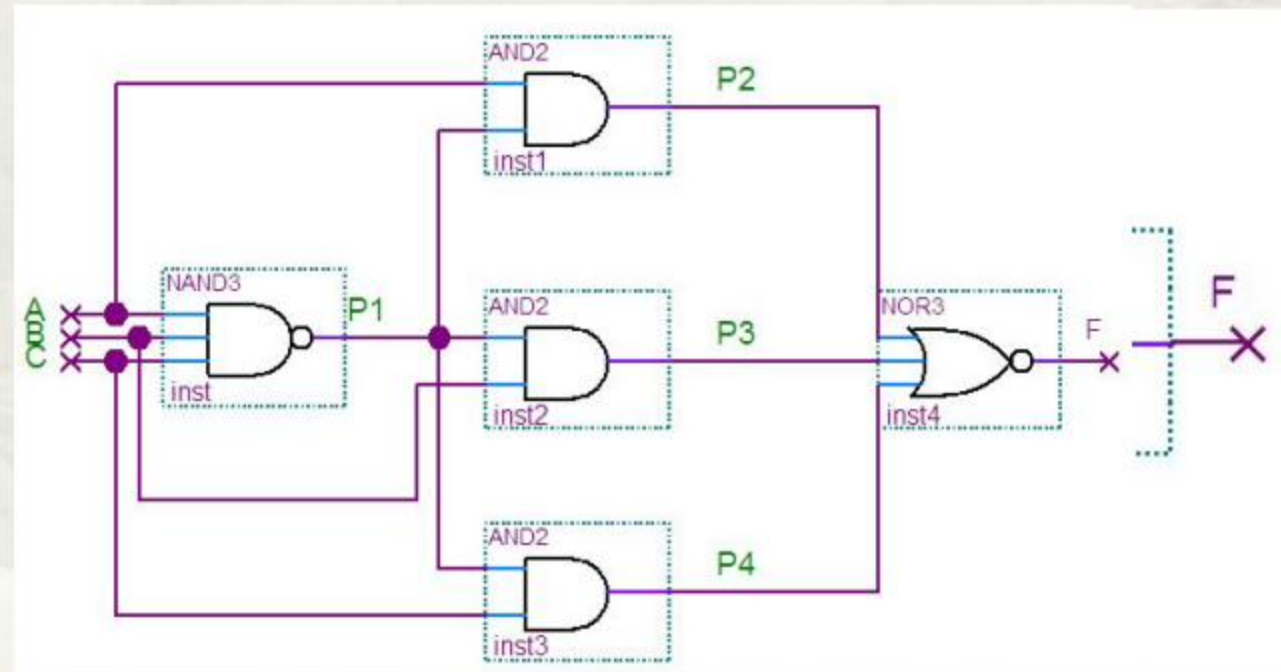
- 4. Conclusion: designed to judge the equivalence of inputs

Done?

- Compare the two:

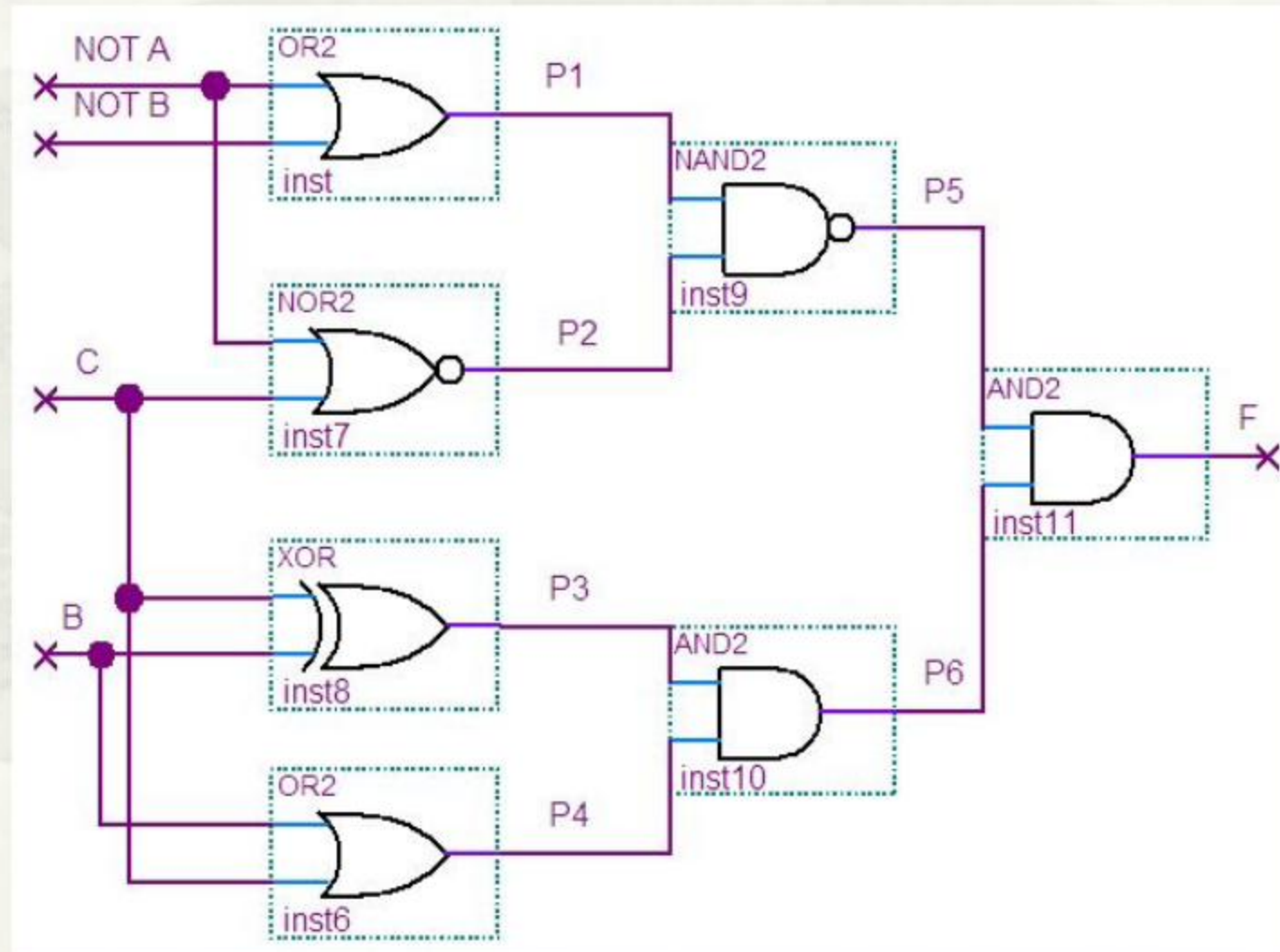
$$F = \overline{P_2 + P_3 + P_4} = \overline{A \cdot \overline{ABC} + B \cdot \overline{ABC} + C \cdot \overline{ABC}}$$

$$F = \overline{\overline{ABC}} + ABC$$



Practice

- Analysis of the circuit given below



3.4 **Design** of basic Combinational Circuits

3.4.1. Approach

- **Step 1:** construct the truth table based on requirements.
- Step 2: the canonical or simplest form.
- Step 3: draw the circuit diagram.

3.4.2. Examples

□ E.g.1. (single output) Implementation of a 3-input voting circuit.

Step 1: construct the truth table.

(1) Number of variables and what do they represent?

(2) Number of function(s) and what does/do it/they

mean?

let x_1, x_2, x_3 be the 3 inputs representing the 3 voters' choice.

Assume $x_1, x_2, x_3 = 1$ representing voting for a proposal;

= 0 representing voting against it.

let f be the output corresponding to the voting result.

Assume $f = 1$ representing the proposal's being voted through;

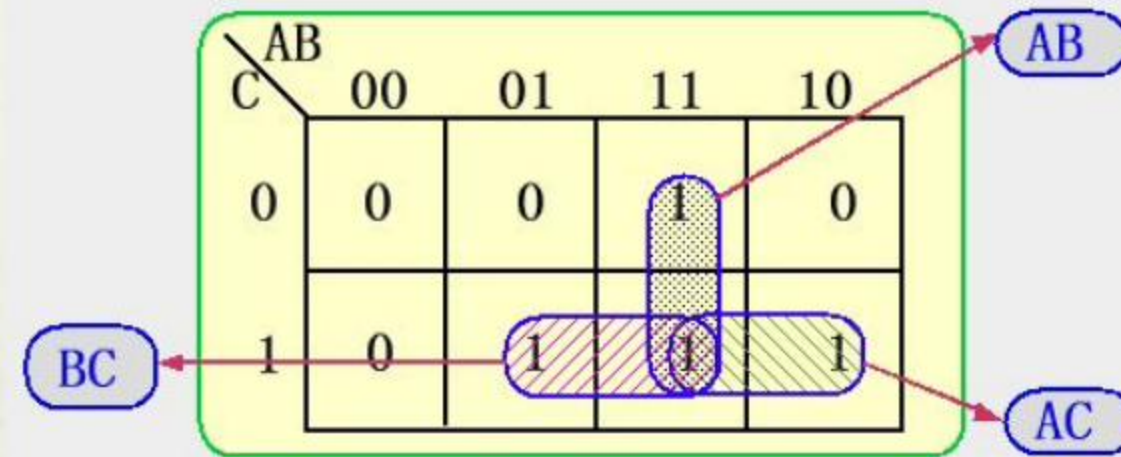
$f = 0$ representing the proposal's being vetoed.

-
- Build the truth table as follows

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

- Step 2: get the canonical SOP and simplify it.

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

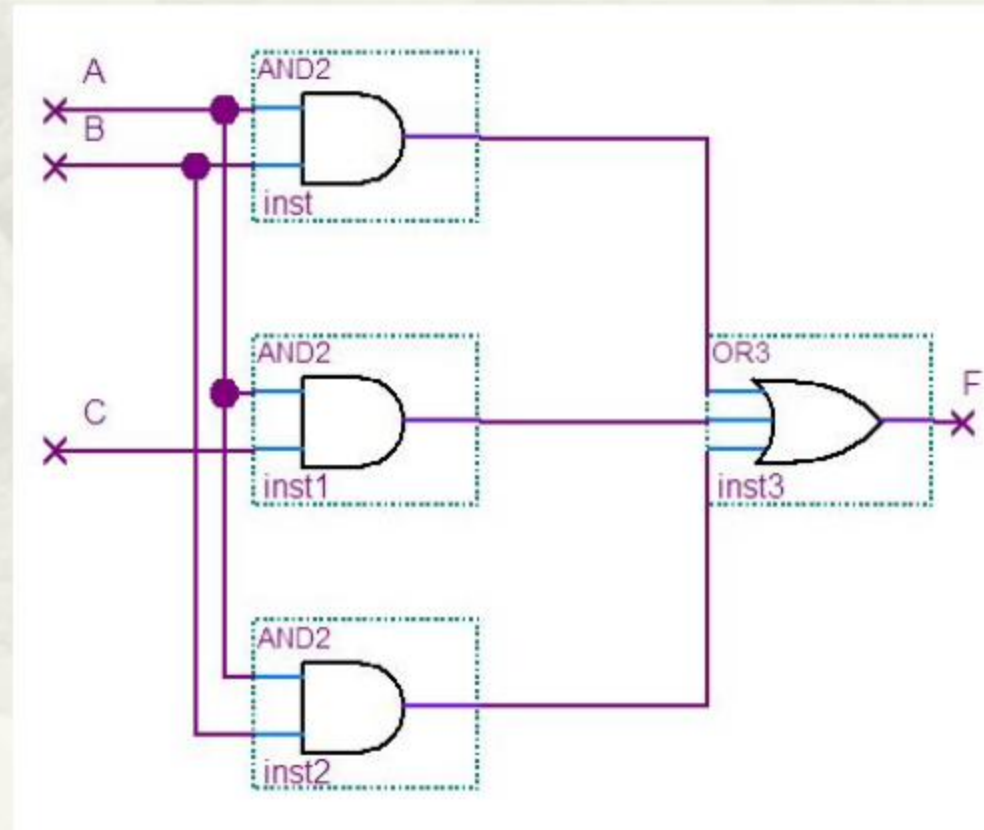


$$f = \sum m(3,5,6,7)$$

$$f = AB + AC + BC$$

- Step 3: draw the circuit diagram based on the simplest SOP.

$$f = AB + AC + BC$$



□ **E.g.2. (multi-output) Implementation of a coded lock.**

□ Logical functions:

- No response if A is solely pressed.
- Unlocked if key A and B, or A and C or A,B,C are pressed simultaneously.
- Alarm otherwise.

Step 1: construct the truth table.

- (1) Number of inputs and what do they represent?
- (2) Number of outputs and what does/do it/they mean?

*Assume $A, B, C = 1$ representing being pressed;
 $= 0$ representing being released.*

let U be an output corresponding to whether it's unlocked.

Assume $U = 1$ if unlocked; $U = 0$ if locked.

let Alarm be another output corresponding to whether it sounds alarms.

Assume $A = 1$ if it does; $U = 0$ if not.

The truth table:

A	B	C	U	Alarm
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	0	1
1	0	0	0	0
1	0	1	1	0
1	1	0	1	0
1	1	1	1	0

Step 2: get the canonical SOPs and simplify them:

$$U = \sum m(5, 6, 7)$$

		A B			
		00	01	1	10
C	0	0	0	1	0
	1	0	0	1	1

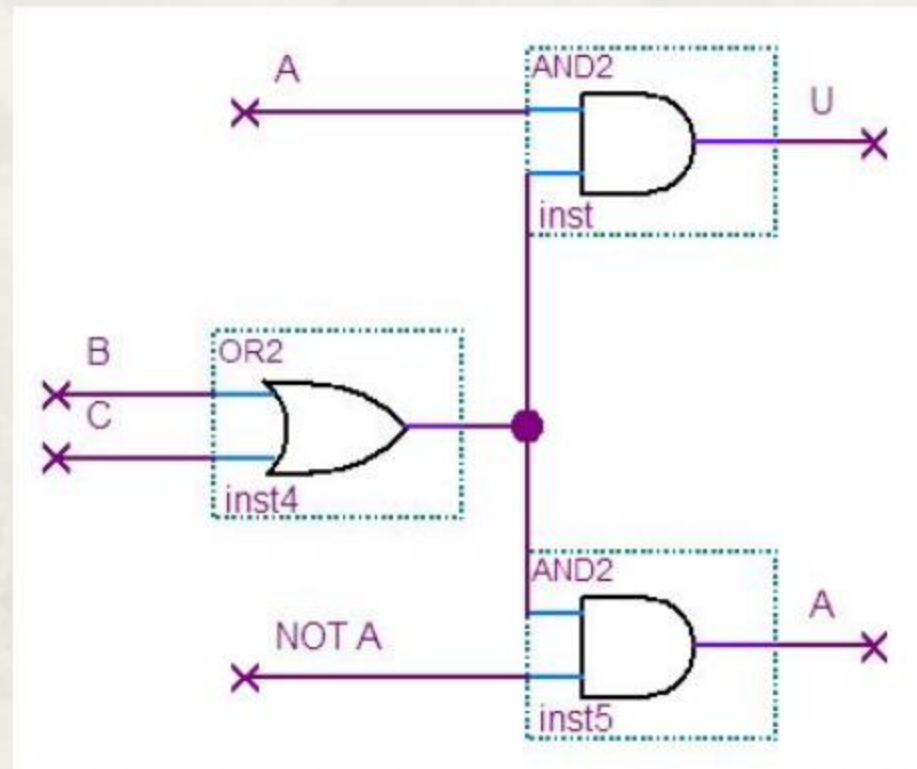
$$U = AB + AC$$

$$A = \sum m(1, 2, 3)$$

		A B			
		00	01	1	10
C	0	0	1	1	0
	1	1	1	0	0

$$A = \overline{A}B + \overline{A}C$$

Step 3: the circuit diagram as follows:



□ **E.g.3. (incompletely specified) Implementation of a comparator.**

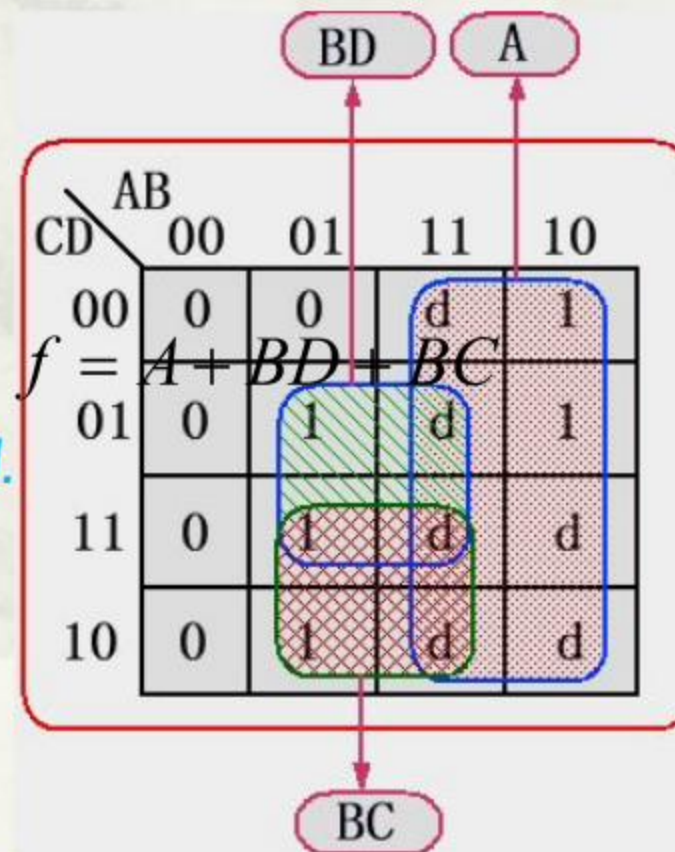
□ Logical functions:

- With BCD codes as inputs.
- Output is 1 when the code >4 and <10 , otherwise 0.

construct the K-map directly and simplify it.

Write simplest SOP:

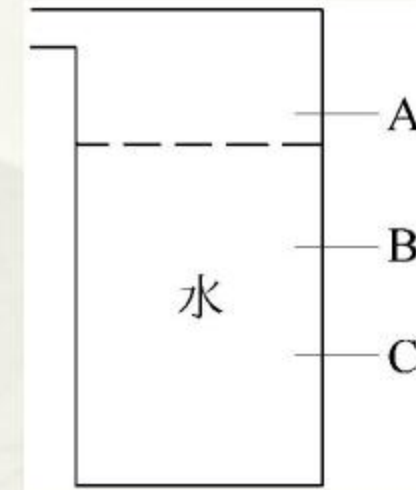
The diagram omitted.



□ **E.g.4. (comprehensive) Design of a WLI(Water Level Indicator).**

□ Logical functions:

- 3 levels: level A, B, C.
- Green light on if the level is between A and B(normal)
- Yellow light on if between B and C or above A(anomalous)
- Red light on if below C(dangerous)



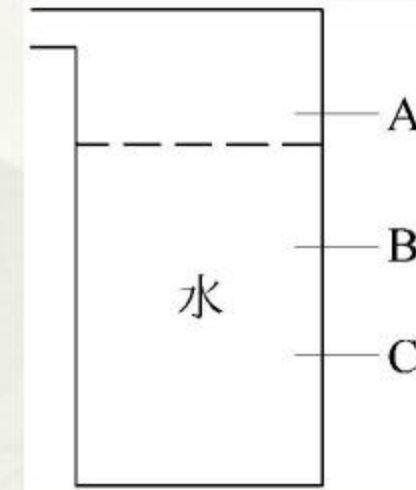
- Step 1: Variables and functions and to which conditions do they correspond?

Assume $A, B, C = 1$ representing the level being above it; $A, B, C = 0$ representing the contrary.

Let G, Y, R be the pilot lights

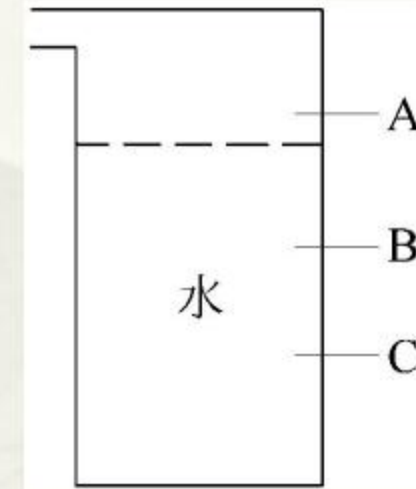
Assume $A, B, C = 1$ representing lights on;

$A, B, C = 0$ representing lights off.



□ The truth table:

A	B	C	G	Y	R
0	0	0	0	0	1
0	0	1	0	1	0
0	1	0	<i>d</i>	<i>d</i>	<i>d</i>
0	1	1	1	0	0
1	0	0	<i>d</i>	<i>d</i>	<i>d</i>
1	0	1	<i>d</i>	<i>d</i>	<i>d</i>
1	1	0	<i>d</i>	<i>d</i>	<i>d</i>
1	1	1	0	1	0



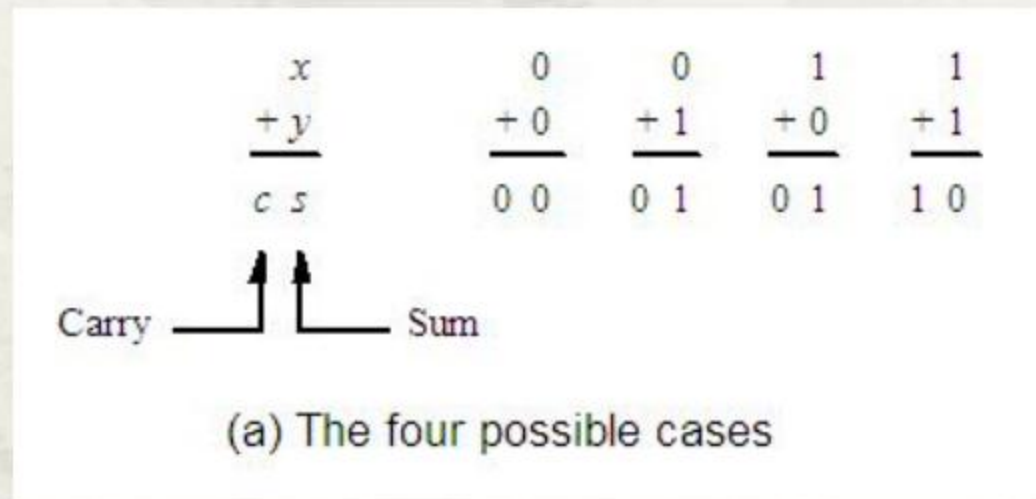
Successive implementation omitted.

3.5 Combinational-Circuit Building Blocks

3.5.1 Adders

1. Half adder(HA)

- Perform addition of 2 1-bit binary numbers.



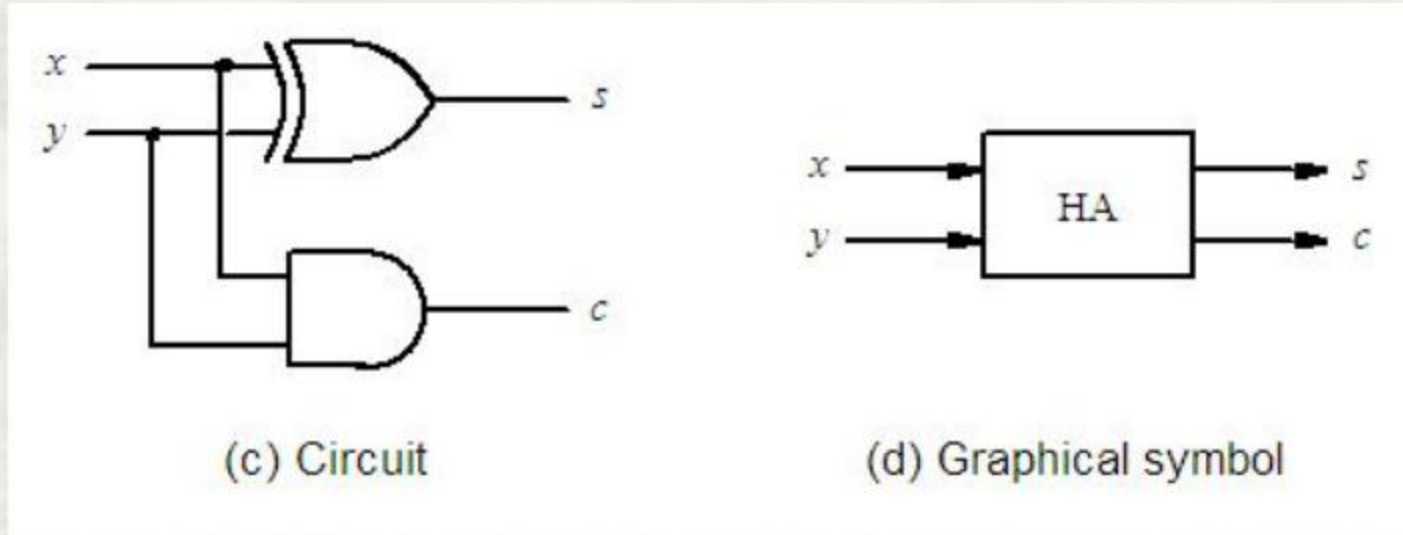
x	y	Carry	Sum
		c	s
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

(b) Truth table

$$s = \bar{x}y + x\bar{y} = x \oplus y$$

$$c = xy$$

□ The circuit diagram



What about multi-bit addition?
E.g. 0110+0111

2. Full adder(FA)

- Perform addition of 2 1-bit binary numbers with carry-in from lower bit position involved.

c_i	x_i	y_i	c_{i+1}	s_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

(a) Truth table

□ Derivation of s_i & c_{i+1}

$\begin{array}{c} x_i y_i \\ \swarrow \searrow \\ c_i \end{array}$		$x_i y_i$			
		00	01	11	10
c_i	0		1		1
	1	1		1	

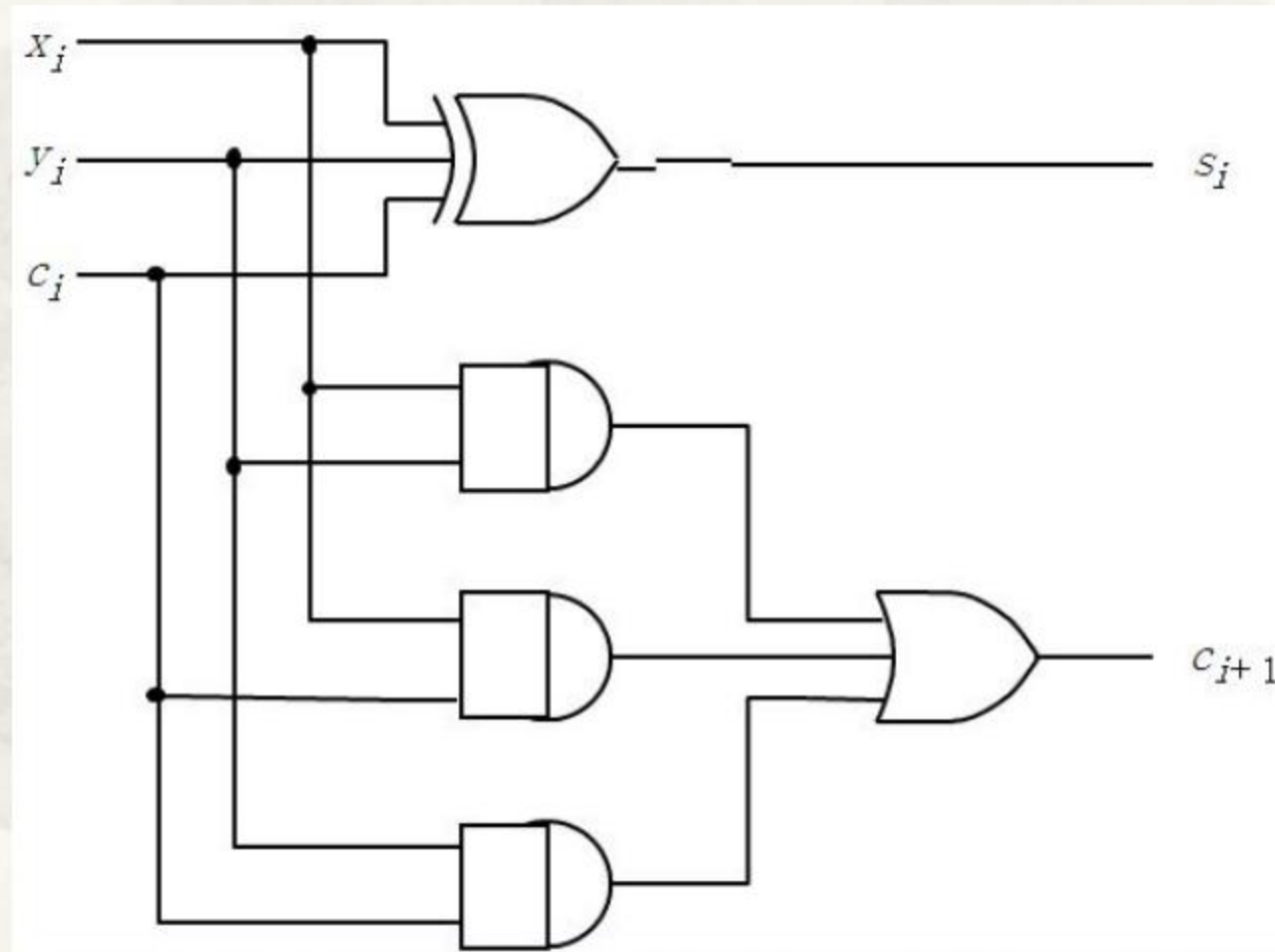
$\begin{array}{c} x_i y_i \\ \swarrow \searrow \\ c_i \end{array}$		$x_i y_i$			
		00	01	11	10
c_i	0			1	
	1		1	1	1

$$\begin{aligned}
 s_i &= \overline{x_i} \overline{y_i} c_i + \overline{x_i} y_i \overline{c_i} + x_i \overline{y_i} \overline{c_i} + x_i y_i c_i \\
 &= (\overline{x_i} \overline{y_i} + x_i y_i) c_i + (\overline{x_i} y_i + x_i \overline{y_i}) \overline{c_i} \\
 &= \overline{(x_i \oplus y_i)} c_i + (x_i \oplus y_i) \overline{c_i} \\
 &= x_i \oplus y_i \oplus c_i
 \end{aligned}$$

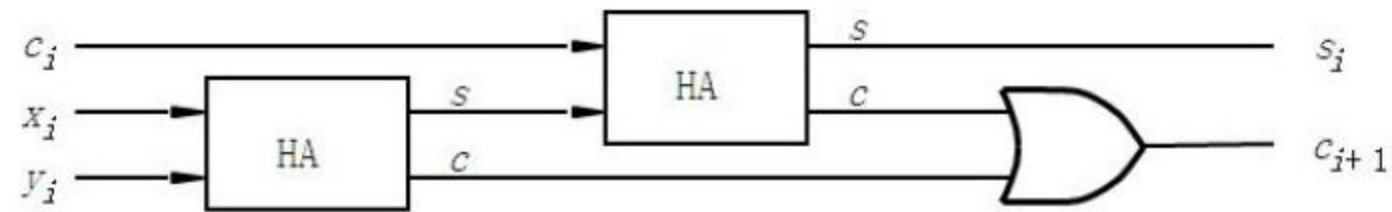
$$c_{i+1} = x_i y_i + x_i c_i + y_i c_i \quad (1)$$

$$\begin{aligned}
 c_{i+1} &= \overline{x_i} y_i c_i + x_i \overline{y_i} c_i + x_i y_i \overline{c_i} + x_i y_i c_i \\
 &= (x_i \oplus y_i) c_i + x_i y_i \quad (2)
 \end{aligned}$$

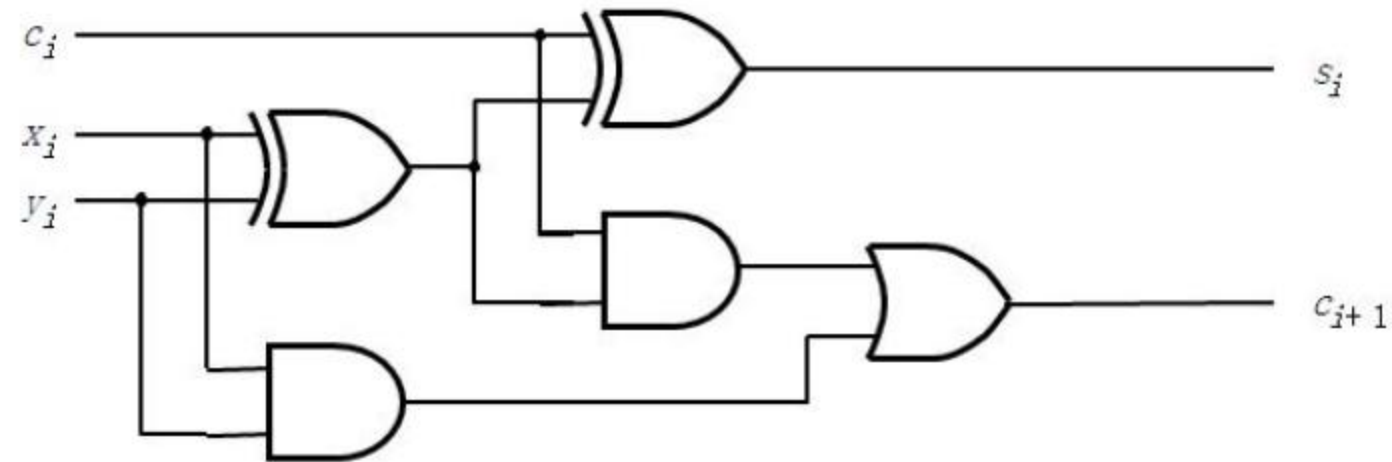
- Circuit 1 (according to c_{i+1} 's expression (1))



□ Circuit 2: modular design(with HAs as components)

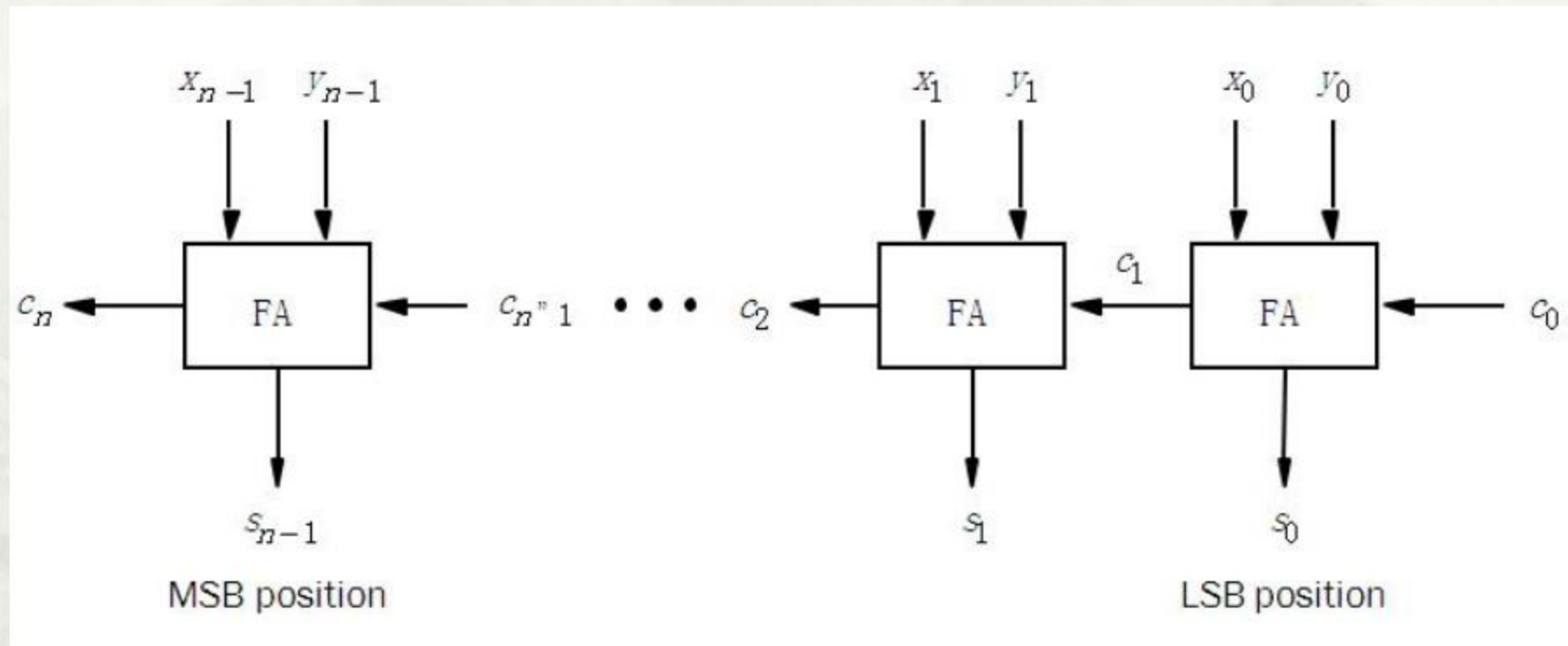


(a) Block diagram



(b) Detailed diagram

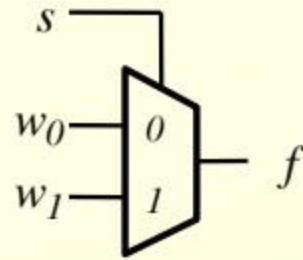
□ Linking FAs to implement multi-bit addition



3.5.2. Multiplexer

- Components of a 2^n -to-1 multiplexer
 - 2^n data inputs
 - n select inputs
 - 1 data output
- Functionality:
 - Passes one of inputs to the only output selected by the values of select inputs.

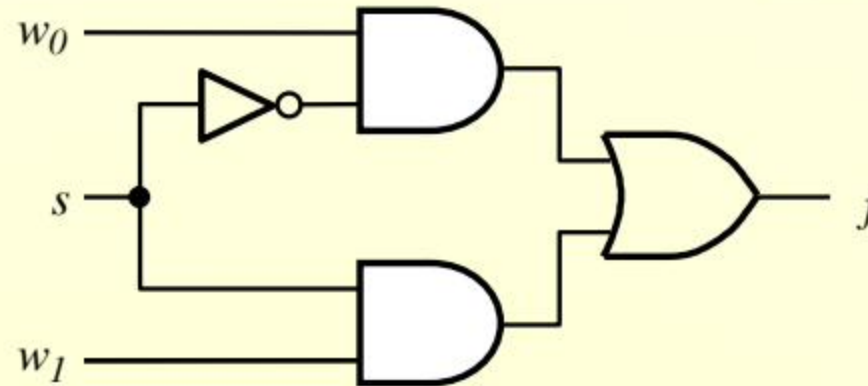
□ A 2-to-1 multiplexer



(a) Graphical symbol

s	f
0	w_0
1	w_1

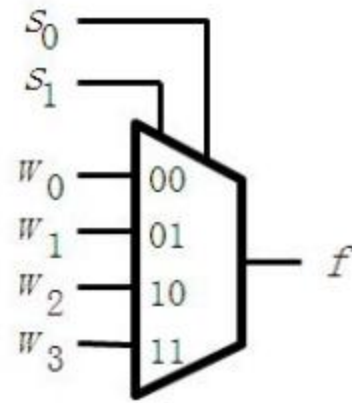
(b) Truth table



(c) Sum-of-products circuit

$$f = \bar{s}w_0 + sw_1 = \sum_{i=0}^1 m_i w_i$$

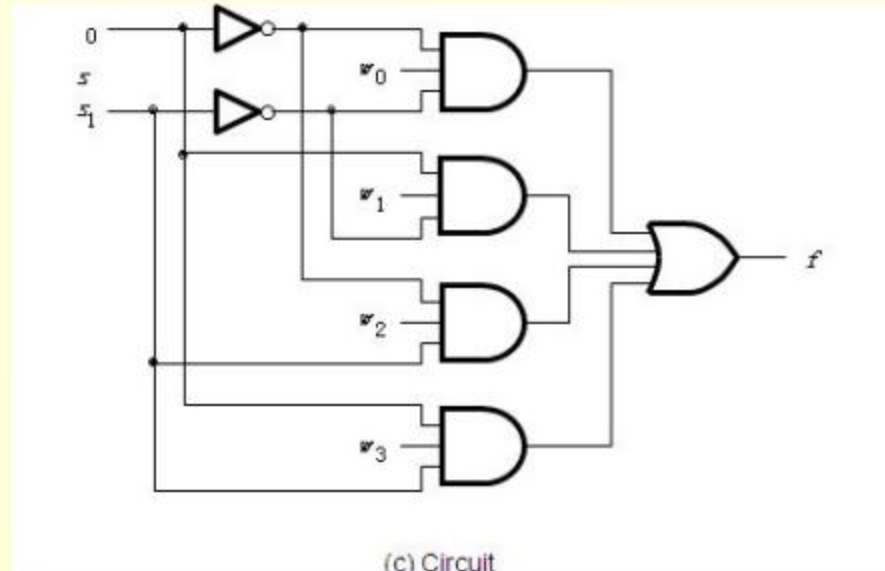
□ A 4-to-1 multiplexer



(a) Graphic symbol

s_1	s_0	f
0	0	w_0
0	1	w_1
1	0	w_2
1	1	w_3

(b) Truth table



(c) Circuit

$$f = \overline{s_1}\overline{s_0}w_0 + \overline{s_1}s_0w_1 + s_1\overline{s_0}w_2 + s_1s_0w_3 = \sum_0^3 m_i w_i$$

It follows that:

- For a 8-to-1 multiplexer :

$$\begin{aligned} f &= \overline{s_2}\overline{s_1}\overline{s_0}w_0 + \overline{s_2}\overline{s_1}s_0w_1 + \overline{s_2}s_1\overline{s_0}w_2 + \overline{s_2}s_1s_0w_3 + \\ &= \overline{s_2}\overline{s_1}\overline{s_0}w_4 + \overline{s_2}\overline{s_1}s_0w_5 + \overline{s_2}s_1\overline{s_0}w_6 + \overline{s_2}s_1s_0w_7 \\ &= \sum_0^7 m_i w_i \end{aligned}$$

- For any multiplexer with n selected inputs:

$$f = \sum_0^{2^n-1} m_i w_i$$

- A multiplexer's output is relevant to the canonical SOP.

Realizing Logic Functions Using Multiplexers

- Approach:
 - Find the canonical SOP of a function
 - List the output expression of the MUX to be used
 - Compare them.
 - Let them equal by setting values for data inputs.
 - Draw the circuit.

E.g.1: Implementation of **XOR2** using a **4-to-1** multiplexer

The function to be implemented:

$$f_1 = \underline{\overline{x_1}x_0} + \underline{x_1\overline{x_0}}$$

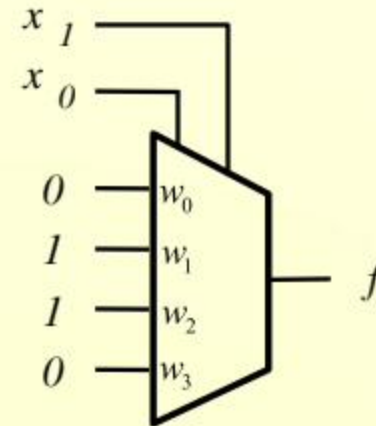
The function of 4-to-1 multiplexer:

$$f_2 = \overline{s_1}\overline{s_0}w_0 + \underline{\overline{s_1}s_0}w_1 + \underline{s_1\overline{s_0}}w_2 + s_1s_0w_3$$

$f_2 = f_1$ by connecting $x_1 \rightarrow s_1, x_0 \rightarrow s_0$;

letting $w_0 = w_3 = 0$ and $w_1 = w_2 = 1$

x_1	x_0	f
0	0	0
0	1	1
1	0	1
1	1	0



A more efficient realization using a 2-to-1 multiplexer:

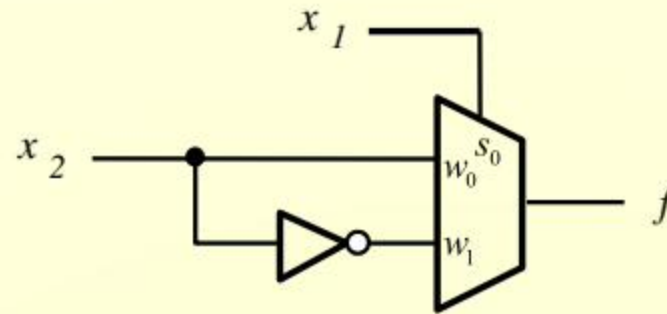
The function to be implemented:

$$f_1 = \underline{\overline{x_1}}x_2 + \underline{x_1}\overline{x_2}$$

The function of the 2-to-1 multiplexer:

$$f_2 = \underline{\overline{s}}w_0 + \underline{s}w_1$$

$f_2 = f_1$ by linking $x_1 \rightarrow s$; $x_2 \rightarrow w_0$; $\overline{x_2} \rightarrow w_1$



Circuit
t

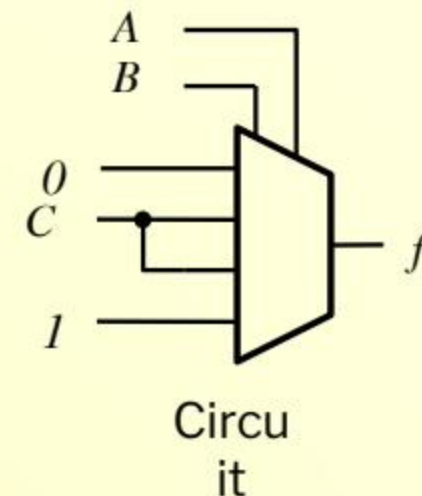
E.g. 2: Implementation of the 3-input majority function using a 4-to-1 multiplexer.

$$\text{The multiplexer : } f_2 = \overline{s_1}\overline{s_0}w_0 + \overline{s_1}s_0w_1 + s_1\overline{s_0}w_2 + s_1s_0w_3$$

$$\begin{aligned} \text{The 3-input majority : } f_1 &= \sum m(3,5,6,7) \\ &= \overline{A}BC + A\overline{B}C + AB\overline{C} + ABC \\ &= \overline{AB} \cdot 0 + \overline{A}BC + A\overline{B}C + AB \cdot 1 \end{aligned}$$

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

$f_2 = f_1$ by linking $x_1 \rightarrow s_1$; $x_2 \rightarrow s_0$ and
setting the 4 inputs as 0, x_3 , x_3 and 1

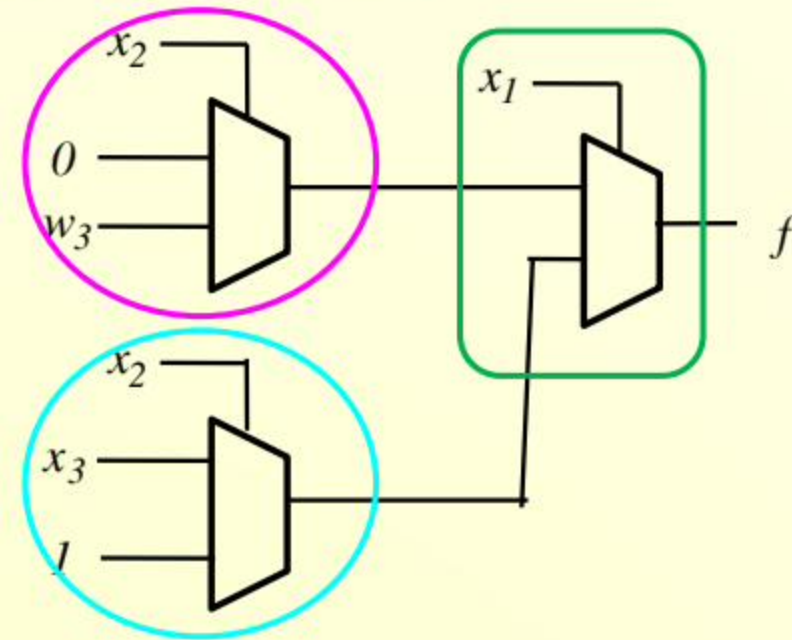


Consider: How about using 2-to-1 multiplexer?

w_1	w_2	w_3	f
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

(a) Truth table

$$\begin{aligned}
 f &= \bar{x}_1 x_2 x_3 + x_1 \bar{x}_2 x_3 + x_1 x_2 \bar{x}_3 + x_1 x_2 x_3 \\
 &= \bar{x}_1 x_2 x_3 + x_1 (\bar{x}_2 x_3 + x_2 \bar{x}_3 + x_2 w_3) \\
 &= \bar{x}_1 (x_2 \cdot 0 + x_2 x_3) + x_1 (x_2 x_3 + x_2 \cdot 1)
 \end{aligned}$$



(b) Circuit

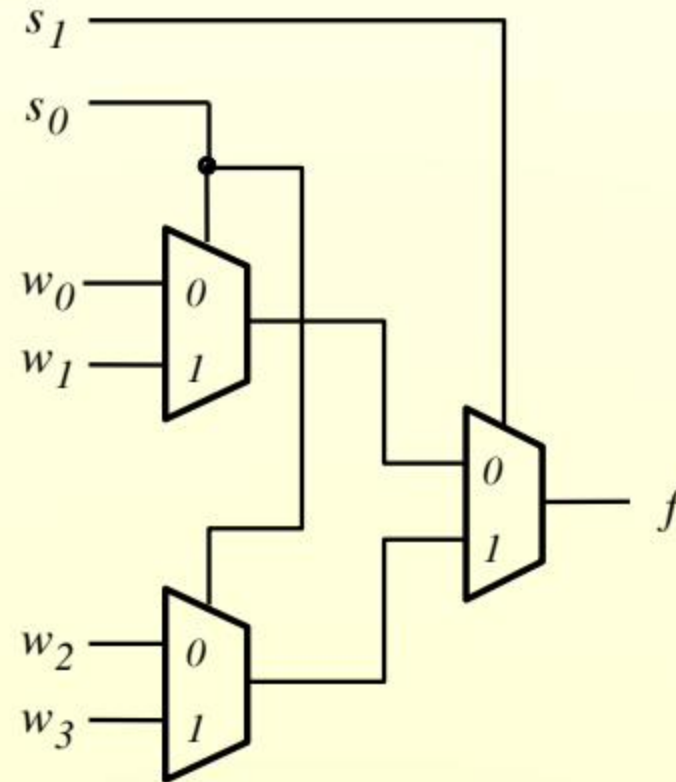
Exercise:

Implement Full adder using multiplexers.

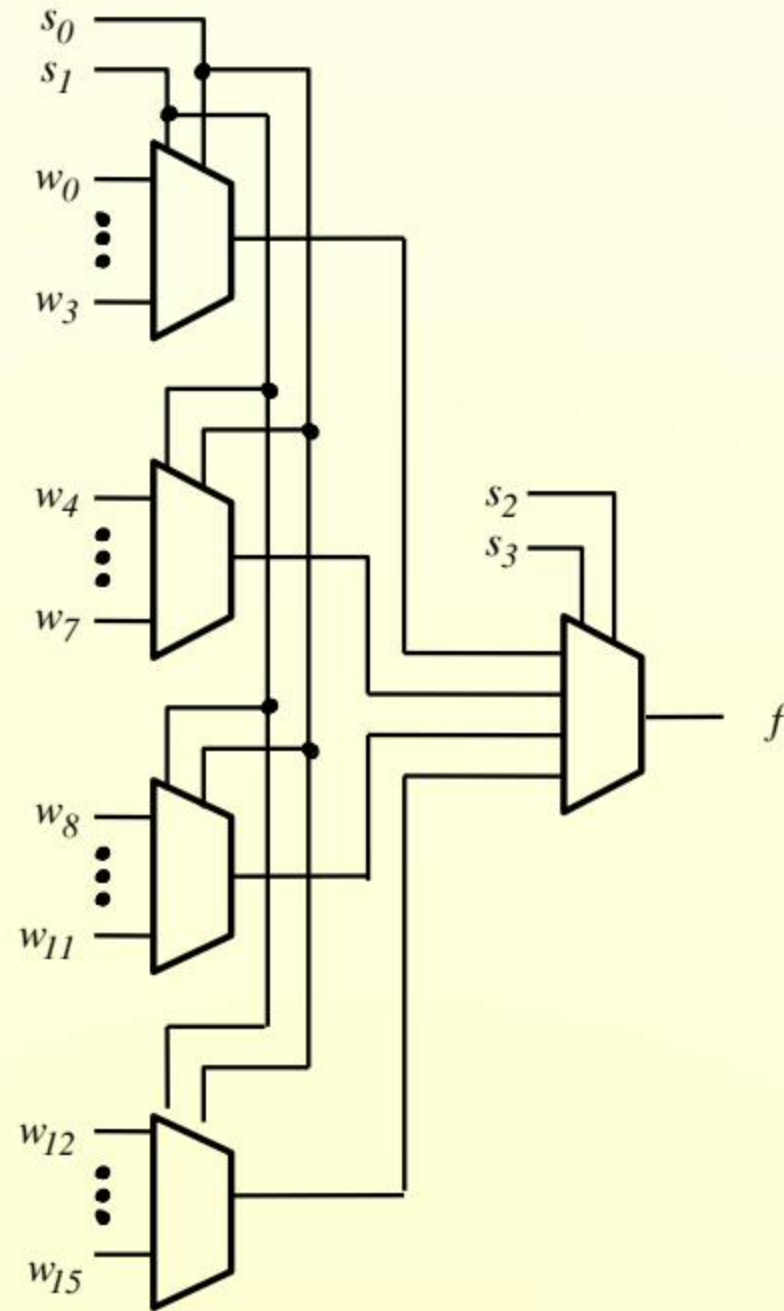
c_i	x_i	y_i	c_{i+1}	s_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

(a) Truth table

Modular design: Build a larger multiplexer with small ones



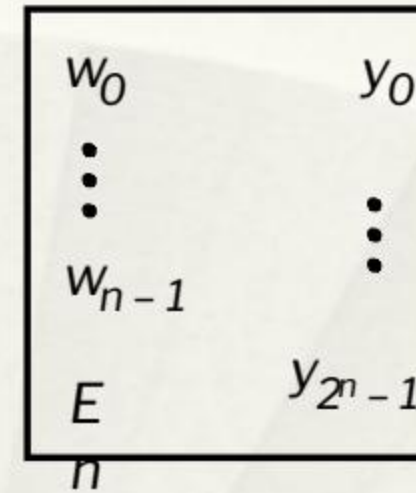
Three 2-to-1 multiplexers make up a 4-to-1 multiplexer.



A 16-to-1 multiplexer consisting of five 4-to-1 ones

3.5.3. Decoders

- n inputs
- 2^n outputs
- An En(able) input

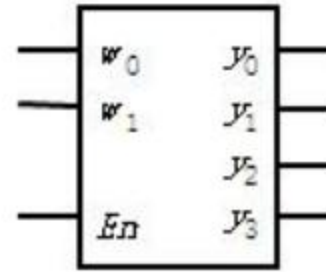


- Functionality
 - **One-hot encoded outputs:** Only 1 output is asserted(active high or active low) under the control of the values of inputs if enabled.
 - No output is active (asserted) if disabled.

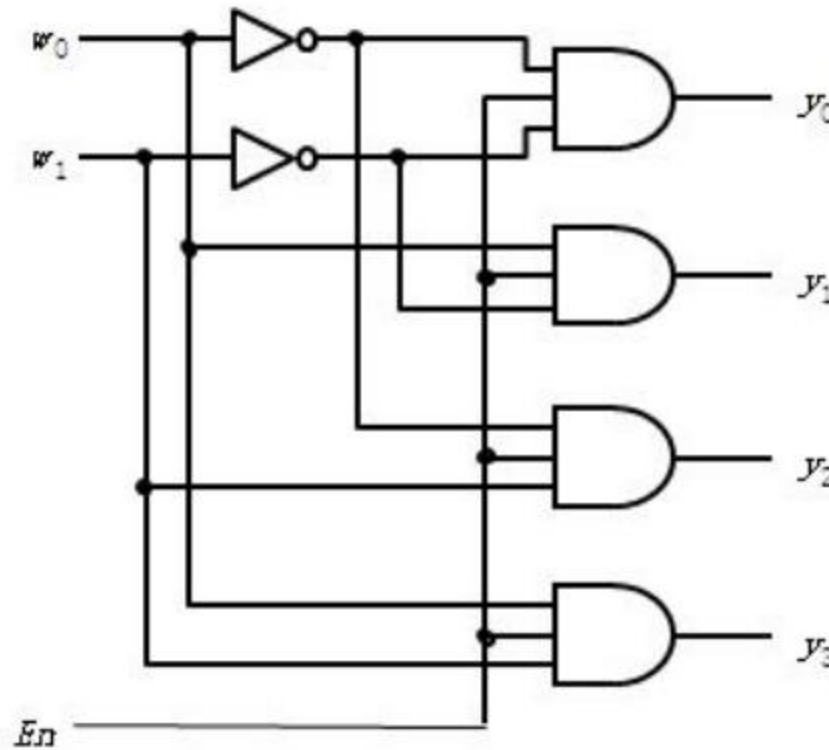
A 2-to-4 decoder

E_n	w_1	w_0	y_0	y_1	y_2	y_3
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1
0	x	x	0	0	0	0

(a) Truth table (active-high)



(b) Graphical symbol



(c) Logic circuit

Regardless of E_n , it gives that:

$$y_0 = \overline{w_1} \overline{w_0} = m_0$$

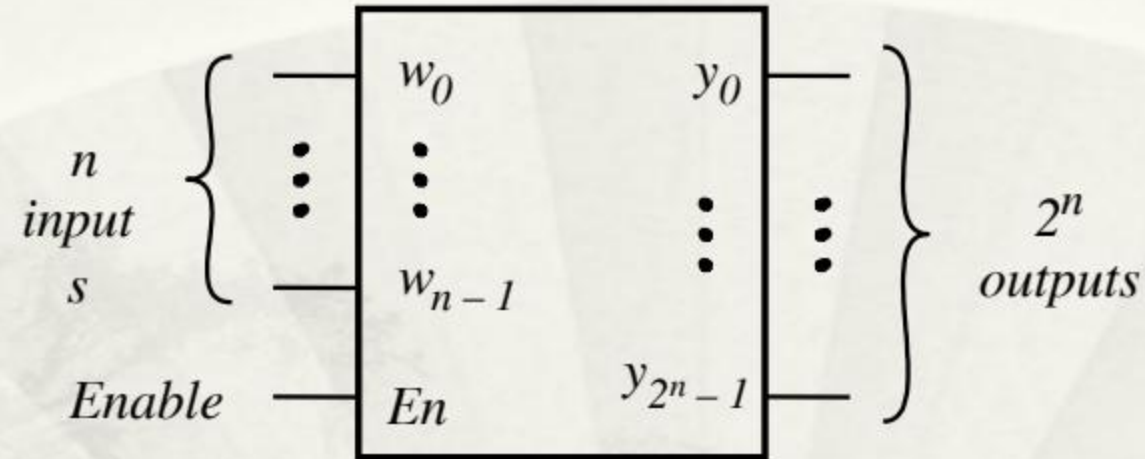
$$y_1 = \overline{w_1} w_0 = m_1$$

$$y_2 = w_1 \overline{w_0} = m_2$$

$$y_3 = w_1 w_0 = m_3$$

Every output matches every m_i with inputs as variables!

Any n -to- 2^n decoder

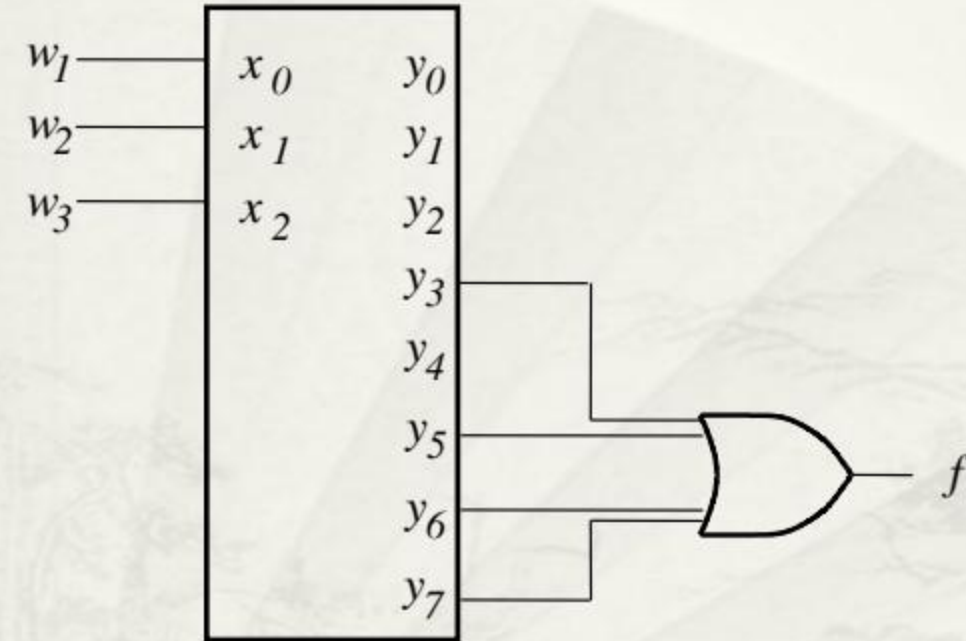


One-to-one correspondence between outputs & minterms with inputs as variables.

Implementation of functions with decoders

e.g.: 3-input majority built by a 3-to-8 decoder

w_1	w_2	w_3	f
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

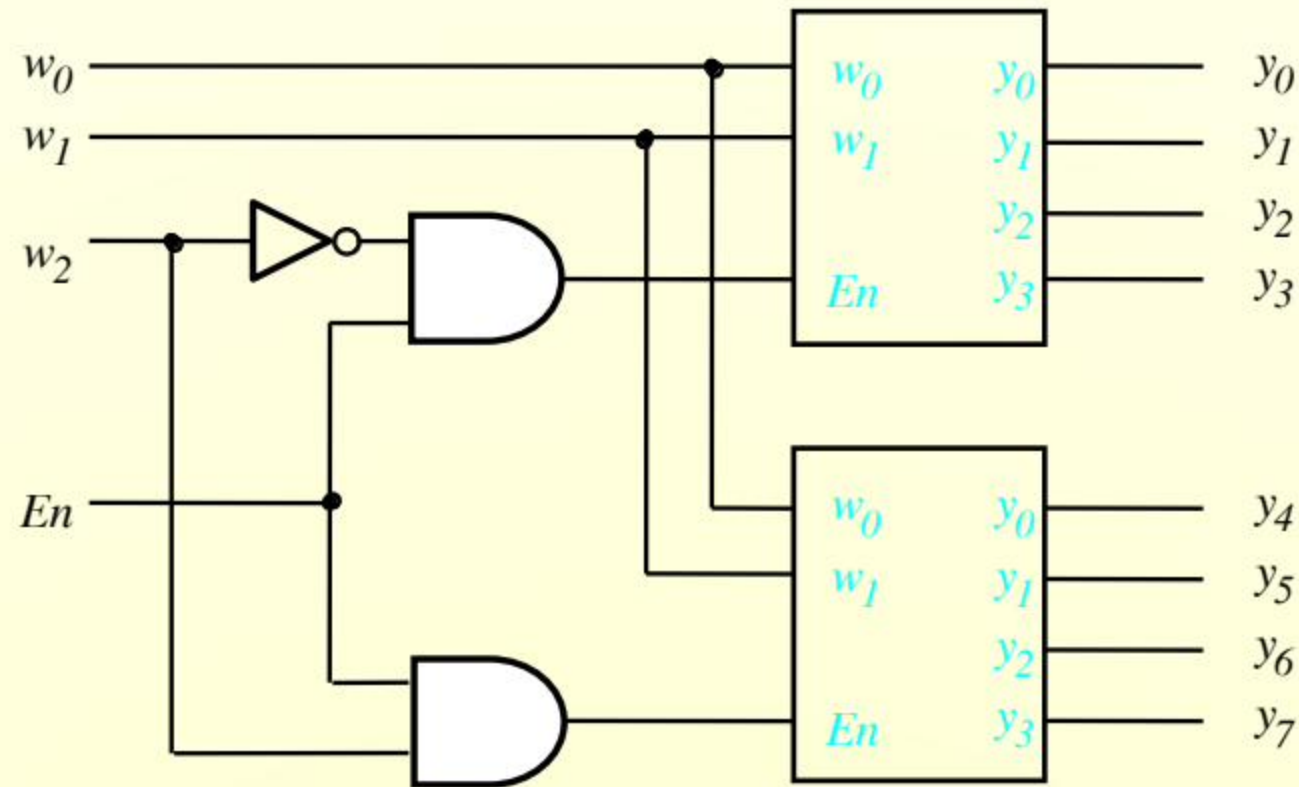


$$f = \sum m(3, 5, 6, 7)$$

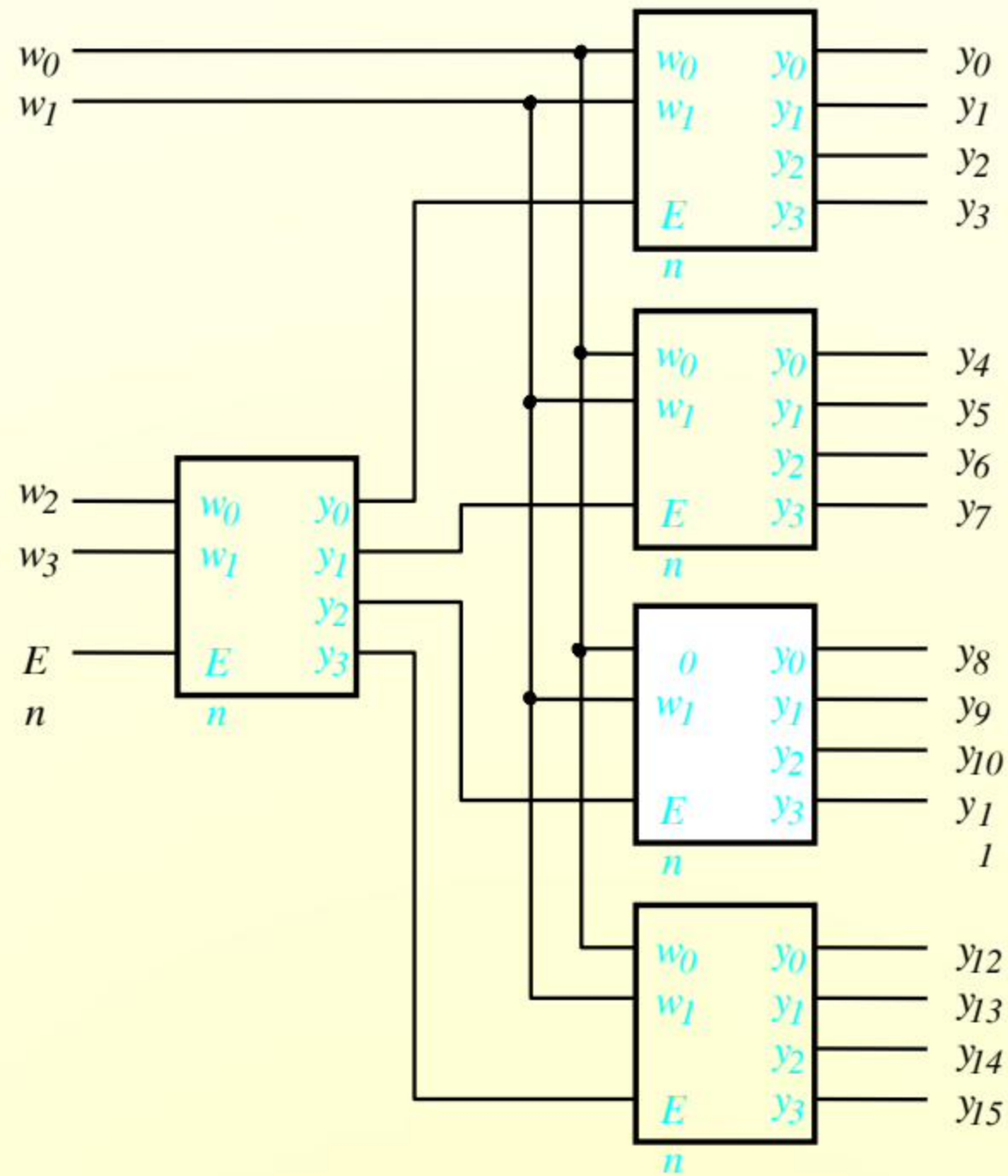
Practice:

Implement Full adder using 3-to-8 decoder.

Hierarchy of decoders



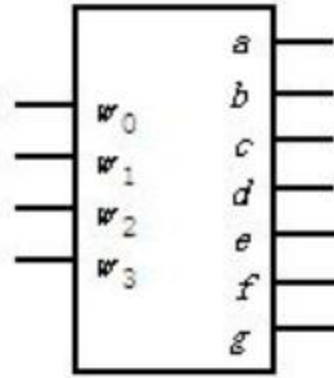
A 3-to-8 decoder using two 2-to-4 decoders.



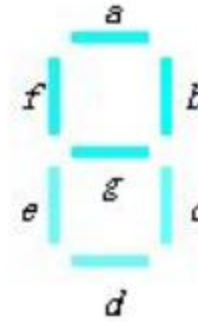
A 4-to-16 decoder built using a **decoder tree**.

3.5.4 Code convertors

- A typical example: **BCD-to-7-segment decoder**



(a) Code converter



(b) 7-segment display

w_3	w_2	w_1	w_0	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1

(c) Truth table